

Protocolos de transmisión confiable

Taller de Redes

25 de agosto, 2026

1. Disclaimer

Al igual que en el taller anterior, para la comunicación estamos usando sockets UDP, algo que se verá más adelante en la materia. Lo ideal sería usar paquetes Ethernet, pero necesitaríamos acceso root para eso. Otro punto a favor de usar UDP es que nos permite tener cliente y servidor en la misma computadora.

En la vida real, el protocolo sliding window funciona mucho más rápido que el stop and wait. Sin embargo, en este taller vamos a transmitir los frames sobre UDP en una máquina local. Este canal tiene un tiempo de propagación despreciable y una velocidad de transmisión altísima, por lo que las ventajas del protocolo sliding window no serán observables. Para este taller nos vamos a enfocar en la correcta implementación del protocolo y no en medir su eficiencia, que requeriría usar herramientas que introduciremos en los talleres siguientes.

2. Enunciado

Vamos a implementar diferentes protocolos para transferir ficheros a través de la red. Para transferir un fichero se corren dos programas:

- El servidor (`PROTOCOL_server_recv.py`), que recibe frames y guarda lo que recibe en un fichero nuevo
- El cliente (`PROTOCOL_client_send.py`), que lee de un fichero existente y manda frames al servidor transmitiendo su contenido

Para enviar ficheros, los protocolos separan el fichero en pedazos (chunks) y los encapsulan dentro de frames de largo fijo. Un último frame se envía activando la opción `is_last` del header para señalar al receptor que no hay más datos para transferir. El fichero `common.py` tiene más detalles sobre el formato de los frames para cada protocolo.

Vamos a usar tres protocolos diferentes:

- Dumb, que manda frames sin verificar si se recibieron
- Stop and wait, que espera un acknowledge del receptor antes de seguir transmitiendo
- Sliding window, manda de a varios frames a la vez pero verificando que se hayan recibido correctamente

2.1. Análisis del protocolo dumb

- a) Ver el código del cliente y servidor para el protocolo dumb (ficheros `dumb_client_send.py` y `dumb_server_recv.py` respectivamente).
- b) Probar transmitir el fichero `tux.bmp` usando este protocolo.

c) Visualizar el BMP recibido. ¿Es igual al original? En caso de que no, comparar el tamaño del fichero original con el recibido, y tratar de entender qué causó el problema.

d) ¿Qué condiciones debería cumplir el canal por el que se envían frames para que la transmisión usando este protocolo siempre funcione correctamente? Simularlas modificando los parámetros y/o el código del cliente y/o servidor.

2.2. Análisis y mejoras al protocolo stop and wait

- a) Repetir los pasos de la sección 2.1 usando el protocolo stop and wait
- b) Modificar el protocolo para incluir un número de secuencia cuyos valores posibles sean 0 o 1. Usar como referencia el protocolo usado en el libro de Peterson.
- c) Comprobar que una vez modificado el protocolo, la transmisión del fichero `tux.bmp` funciona correctamente
- d) ¿Qué condiciones debería cumplir el canal para que el protocolo modificado funcione correctamente?

2.3. Implementación de cliente para sliding window

- a) Ver brevemente el código de `slidingwindow_client_send.py`, correspondiente al emisor para el protocolo sliding window
- b) Completar `slidingwindow_server_recv.py` para que reciba ficheros usando el protocolo de sliding window.
Sugerencia: basarse en la implementación modificada de stop and wait que hicieron en la sección 2.2. Usar una ventana de tamaño 1 en el receptor para facilitar la implementación.
- c) Transmitir el fichero `tux.bmp` y comprobar que funciona correctamente

2.4. Ejercicios adicionales

Probar cambiar el `CHUNK_SIZE` en `common.py` a diferentes valores y visualizar el BMP recibido cuando se usan protocolos de transmisión no confiables.

Usar la opción `--address` del cliente para transmitir el fichero a otra computadora a través de la red. ¿Cómo se comportan los diferentes protocolos?

Experimentar transmitir imágenes de formatos que no sean BMP usando los protocolos no confiables y observar visualmente el resultado.

3. Bibliografía sugerida

Para el header de los frames usamos el módulo `struct` de Python, que permite convertir entre tipos de datos y bytes. Se puede ver la documentación oficial aquí: <https://docs.python.org/3/library/struct.html>.

Larry L. Peterson and Bruce S. Davie. Computer Networks: A Systems Approach. Capítulo 2.5 Reliable Transmission.